MODUL PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBYEK

MODUL 14
Pengujian, Refactoring, dan Finalisasi Proyek



Disusun oleh:

Nirwana Haidar Hari, S.Pd., M.Kom.

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
PROGRAM STUDI DI LUAR KAMPUS UTAMA SUMENEP
PENS PSDKU SUMENEP

2026

MODUL PRAKTIKUM P14
Pengujian, Refactoring, dan Finalisasi Proyek

Mata Kuliah: Praktikum Pemrograman Berorientasi Obyek
Model Pembelajaran: Project Based Learning
Prasyarat: Aplikasi proyek telah terintegrasi sebagaimana P13.

1. Tujuan Praktikum

Mahasiswa mampu:

- menguji fungsi dan konsistensi data aplikasi;
- menemukan serta memperbaiki kesalahan berdasarkan bukti;
- melakukan refactoring tanpa mengubah perilaku yang sudah benar;
- menyiapkan proyek untuk demonstrasi dan evaluasi.

2. Dasar Teori Singkat

Konsep	Pengertian
Pengujian fungsional	Memeriksa kesesuaian fitur dengan kebutuhan.
Pengujian integrasi	Memeriksa kerja sama antarbagian aplikasi.
Pengujian regresi	Mengulang pengujian setelah perubahan untuk memastikan fitur sebelumnya tetap bekerja.
Refactoring	Memperbaiki struktur kode tanpa mengubah perilaku yang diharapkan.
Class invariant	Aturan yang harus dipertahankan agar state objek tetap valid.

Program yang dapat dijalankan belum tentu benar. Keberhasilan juga ditentukan oleh ketepatan hasil, penanganan kegagalan, dan konsistensi data.

3. Studi Kasus

Gunakan proyek kelompok masing-masing. Untuk contoh perpustakaan, alur utama yang harus bekerja adalah:

Muat data → tambah buku → cari → hapus → simpan → tutup → jalankan kembali.

Aturan yang harus dipertahankan:

- kode buku unik;
- kode dan judul tidak kosong;
- data yang ditolak tidak masuk ke collection;
- kegagalan pemuatan tidak merusak collection sebelumnya;
- perubahan yang berhasil disimpan tersedia saat aplikasi dijalankan kembali.

4. Langkah Praktikum

Langkah 1 — Tetapkan kondisi awal

Sebelum pengujian:

1. Cadangkan file data.
2. Siapkan data awal yang diketahui isinya.
3. Catat jumlah dan identitas data awal.
4. Tentukan hasil yang diharapkan sebelum menjalankan skenario.

Contoh data awal:

B001|Dasar Java

Gunakan salinan data awal untuk setiap skenario yang memerlukan kondisi tersebut agar hasil pengujian tidak saling memengaruhi.

Langkah 2 — Jalankan pengujian

No.	Skenario	Hasil yang diharapkan
1	Tampilkan collection kosong	Pesan koleksi kosong; aplikasi tetap berjalan.
2	Tambah kode baru dengan judul valid	Data bertambah satu.
3	Tambah kode yang sudah terdaftar	Ditolak; data lama tidak berubah.
4	Tambah kode atau judul berisi spasi saja	Ditolak.
5	Tambah judul mengandung |	Ditolak sebelum masuk collection.
6	Cari kode dengan huruf kecil, misalnya b001	Buku ditemukan sesuai aturan pencarian P11.
7	Hapus kode yang tidak terdaftar	Pesan tidak ditemukan; collection tetap.
8	Simpan, tutup, lalu jalankan kembali	Identitas dan isi data tetap sesuai.
9	Muat file dengan baris rusak atau kode duplikat	Ditolak; collection sebelumnya tetap.
10	Keluar saat ada perubahan, lalu batalkan	Aplikasi kembali ke menu; perubahan tetap tersedia.

Sesuaikan skenario dengan domain proyek, misalnya stok kurang, saldo tidak cukup, atau kendaraan sedang disewa.

Langkah 3 — Catat dan perbaiki kesalahan

Gunakan format berikut:

ID	Langkah pemicu	Hasil yang diharapkan	Hasil aktual	Status
BUG-01	Tambah judul mengandung pemisah	Ditolak saat penambahan	Masuk collection, gagal saat disimpan	Belum diperbaiki
…	…	…	…	…

Contoh tersebut merupakan **contoh temuan**, bukan hasil pengujian proyek Anda.

Untuk setiap temuan:

1. Ulangi langkah pemicu.
2. Tentukan sumber kesalahan.
3. Perbaiki pada class yang bertanggung jawab.
4. Jalankan kembali pengujian yang gagal.
5. Periksa fitur terkait melalui pengujian regresi.

Jangan mengisi temuan atau hasil pengujian yang tidak benar-benar terjadi.

Langkah 4 — Lakukan refactoring

Periksa pembagian tanggung jawab berikut:

Bagian	Pemeriksaan dan tindakan
Main	Pecah blok menu panjang menjadi method yang jelas.
Class domain	Pertahankan validasi state di constructor atau method domain.
Class pengelola	Pusatkan pencarian, keunikan kode, dan pengelolaan collection.
Operasi file	Pisahkan ke repository jika kompleksitas proyek memerlukannya.
Penamaan	Gunakan nama yang menjelaskan tujuan variabel dan method.
Kode berulang	Satukan proses yang benar-benar memiliki tanggung jawab sama.

Contoh pemecahan menu:

```
private static void menuCari(
        Scanner input, Perpustakaan perpustakaan) {

    System.out.print("Kode buku: ");
    String kode = input.nextLine().trim();

    Buku hasil = perpustakaan.cariBuku(kode);

    if (hasil == null) {
        System.out.println("Buku tidak ditemukan.");
    } else {
        System.out.println("Judul: " + hasil.getJudul());
    }
}
```

Pemanggilan pada menu utama menjadi:

```
case "3":
    menuCari(input, perpustakaan);
    break;
```

Perhatikan: pemeriksaan kode unik tetap berada di class pengelola, meskipun kode menu dipindahkan.

Langkah 5 — Uji regresi dan perbarui UML

Setelah refactoring:

- ulangi skenario pengujian;
- bandingkan hasil sebelum dan sesudah perubahan;
- perbarui UML sesuai struktur class terakhir;
- pastikan program dapat dikompilasi dari source code yang dikumpulkan.

5. Tugas Proyek Kelompok

Finalisasikan proyek dengan ketentuan:

1. Jalankan minimal **10 skenario pengujian**.
2. Perbaiki kesalahan yang menghambat alur utama atau merusak konsistensi data.
3. Lakukan minimal **dua perbaikan struktur kode**, disertai alasan.
4. Pertahankan penerapan OOP sesuai kebutuhan kasus; jangan menambahkan inheritance hanya untuk memenuhi jumlah class.
5. Siapkan demonstrasi selama **5–7 menit**.

Urutan demonstrasi:

1. Kebutuhan dan pembagian tanggung jawab class.
2. Alur penggunaan utama.
3. Penolakan satu input tidak valid.
4. Penyimpanan dan pemuatan kembali.
5. Satu perbaikan kode beserta hasil pengujiannya.

6. Luaran

Setiap kelompok mengumpulkan:

- source code final;
- UML Class Diagram terbaru;
- contoh file data tanpa informasi pribadi;

- tabel pengujian dengan hasil aktual;
- catatan perbaikan dan refactoring;
- README.md berisi kebutuhan JDK, cara kompilasi, cara menjalankan, serta lokasi file data.

7. Penilaian

Aspek	**Bobot**
Ketepatan fungsi dan konsistensi data	30%
Penerapan OOP dan pembagian tanggung jawab	25%
Kelengkapan pengujian dan bukti hasil	20%
Refactoring dan pengujian regresi	15%
Dokumentasi dan demonstrasi	10%
Total	**100%**

8. Pertanyaan Evaluasi

1. Mengapa program yang berhasil dikompilasi masih perlu diuji?
2. Apa perbedaan perbaikan bug dan refactoring?
3. Mengapa pengujian harus diulang setelah struktur kode diubah?
4. Bagaimana membuktikan operasi yang gagal tidak merusak state objek?

Tugas minggu depan: siapkan versi proyek yang stabil untuk demonstrasi, lengkapi dokumentasi, dan pastikan setiap anggota mampu menjelaskan class serta keputusan desain yang menjadi tanggung jawabnya.